Academic honesty and plagiarism

- Plagiarism is unacceptable.
  - Code you submit must be your own. No copying, adapting, or submitting code you did not create is allowed. Working together and presenting variants of the same file is not acceptable. Here are some specific guidelines to make sure you don't cross the line:
    - Do not exchange programs or program fragments in any form on paper, via e-mail, photos, or by other means.
    - Do not copy solutions from any source, including the web or previous quarters' students.
    - Do not discuss code with other students at the level of detail that will lead to identical programs or program fragments.
    - Do not use chatGTP, co-pilot, or other AI tools
  - Ask me if you are uncertain.
  - **All violations of the academic honesty will be immediately referred to the dean's office.**
- **Plagiarism detection will be applied to all code. It is very good and will catch you!! (20% of a class referred to dean!!)**
- Inadvertent Disclosures: Please do not place class materials on any public site. If you decide to place your work in a public repository (e.g. github), be sure that you mark the permissions so that only you can access it.

Lab 1: Uninformed Search

Due Jan 15, 7:00 pm

The task in this lab is to create a search algorithm to help a robot find its way to a destination. The robot exists in a grid world named "map" of size MAP_WIDTH by MAP_HEIGHT, and it can move in all 4 directions (diagonals not allowed) through empty space cells only by steps of exactly 1 cell distance.

The starting location of the robot is marked in map with a number "2" and the goal with a number "3". The other two values you can find in the map are "1" for walls and "0" for empty space.

Sample map of size MAP_WIDTH 5 and MAP_HEIGHT 5

1	1	1	1	0
0	2	0	0	0
0	1	1	1	1
0	1	1	3	1
0	0	0	0	0

Robot starting location in map[1][1] Goal in map[3][3], upper left location is (0,0)

You are to complete the code for 2 functions:

df_search(map) and **bf_search**(map)

The first one uses depth first search and the second one uses breadth first search to find a path from the robot starting location to the goal. The functions should return a boolean value "true" if the destination was reached and "false" otherwise. An additional condition is to mark the map with a number "4" in all explored cells and with a number "5" in the cells that are part of path found.

To make sure everybody arrives to the same results (very important for the automated grader) you must use the following search order for map[y][x]: First [y][x+1], then [y+1][x], then [y][x-1], and finally [y-1][x] Search order means order nodes are expanded from the frontier.

Don't use any additional python modules or packages. Make sure you are using Python3

Considerations:

● We provided several maps to let you test your solution, but the grading will use a different set. Feel free to create your own test cases. Good performance on the test cases is not a guarantee you will do well on the grading cases.

● Remember to use the provided constants for the map sized and do not hardcode any values because during grading the dimensions of the map can be different.

● The starting location of the robot and the destination are part of the path and should be marked with a “5” in the map.

● The running time of your algorithm cannot be longer than 10 seconds for a 15x15 maps or smaller, otherwise it will fail the grading tests.

● All maps will have a maximum of 1 possible path between the starting location and the destination (to make it easier).

● Unreachable goals are possible.

● There will be no loops in the maps (to make it easier).

● There will always be exactly 1 starting position and 1 goal